section, you'll write the CTE first with SQL and then look at how to translate a CTE into Active Record.

First, write a SQL query without a CTE that finds Drivers created in the last 30 days. From psql connected to the Rideshare database, run this SQL query:

sql/trips_new_drivers.sql

```
-- Drivers created in the last 30 days
SELECT *
FROM users
WHERE created_at >= (NOW() - INTERVAL '30 days');
```

Imagine a Rideshare feature that shows ratings for new Drivers to driver operations managers. To fetch the data, use the rating field on the trips table.

Your CTE will have named query parts so that the query stays organized and maintainable. To populate a lot of rated trips, run bin/rails data_generators:trips_and_requests from your terminal. Once you've got a lot of rated trips available, enter this SQL statement into psql:

sql/trips_top_drivers_cte.sql

```
WITH new_drivers AS (
  SELECT *
  FROM users
  WHERE type = 'Driver'
  AND created_at >= (NOW() - INTERVAL '30 days')
),
rated_trips AS (
  SELECT id, driver_id
  FROM trips
  WHERE rating IS NOT NULL
)
-- Display Driver names and
-- the Average rating for their trips
SELECT
  trips.driver_id,
  CONCAT(users.first_name, ' ', users.last_name) AS driver_name,
  ROUND(AVG(trips.rating), 2) AS avg_rating
FROM trips
JOIN users ON trips.driver_id = users.id
JOIN new_drivers ON trips.driver_id = new_drivers.id
JOIN rated_trips ON trips.id = rated_trips.id
WHERE users.type = 'Driver'
GROUP by 1, 2
ORDER BY 3 DESC
LIMIT 10;
```

Let's break down what's happening in the CTE above. The CTE has named parts new_drivers and rated_trips which are each independent queries being brought together to combine Driver data and Trips data. With the data sources